

Министерство науки и высшего образования Российской Федерации
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ
ТОМСКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ (НИ ТГУ)
Институт прикладной математики и информатики

ОТЧЕТ ПО ЛАБОРАТОРНОЙ РАБОТЕ
по дисциплине «Системы управления базами данных»

Функции агрегации

Бриц Владислав Олегович

Специалитет, 10.05.01,
Направленность «Компьютерная Безопасность»

Руководитель работы
Старший преподаватель кафедры
теоретических основ информатики

_____ Е.Е. Мокина

подпись

«26» ноября 2022 г.

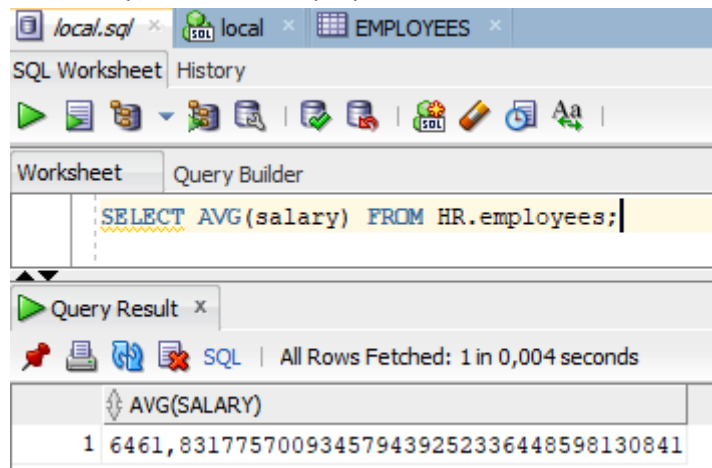
Автор работы
студент группы № 932025

_____ В.О. Бриц

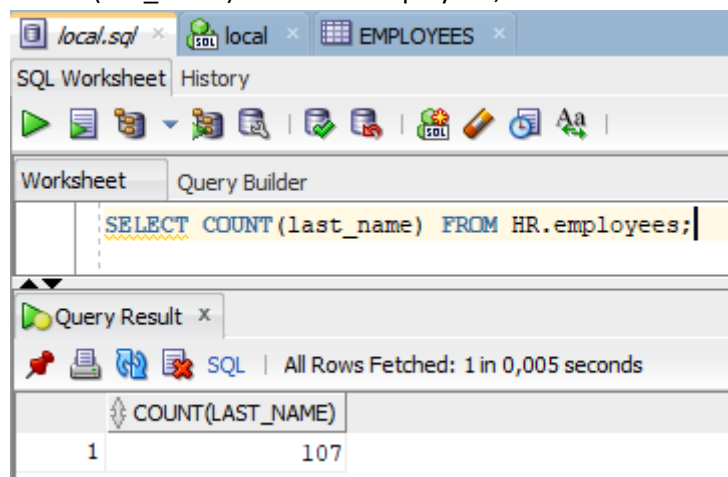
подпись

«26» ноября 2022 г.

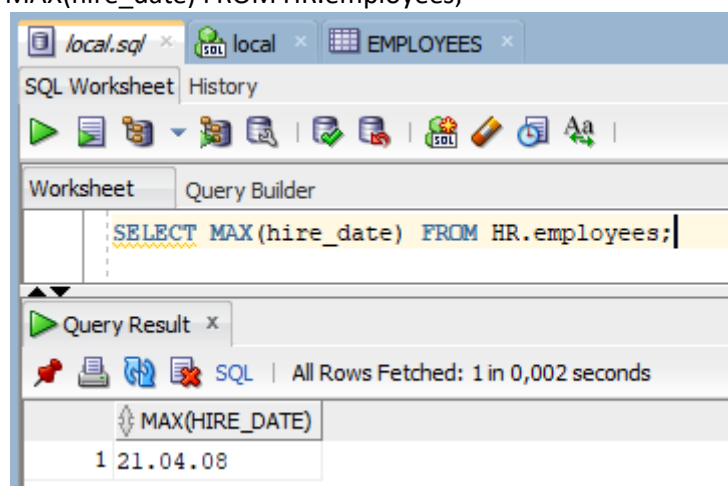
1. Вывести среднюю заработную плату всех сотрудников.
SELECT AVG(salary) FROM HR.employees;



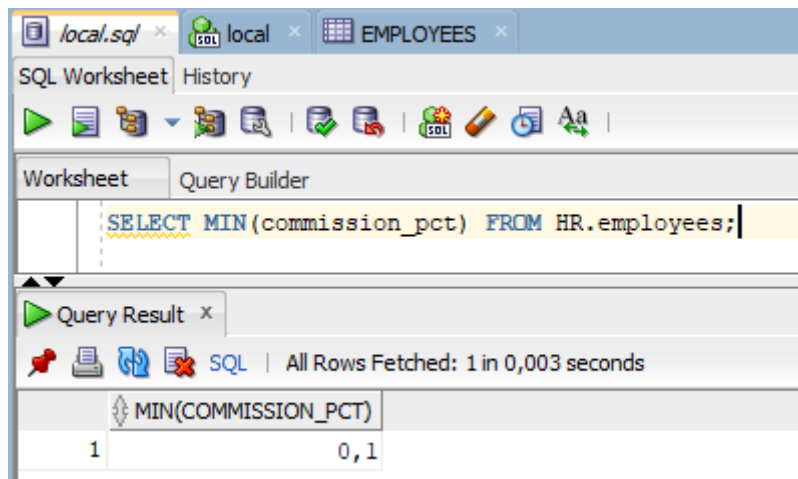
2. Вывести количество сотрудников.
SELECT COUNT(last_name) FROM HR.employees;



3. Вывести дату найма сотрудника, которого наняли позже всех.
SELECT MAX(hire_date) FROM HR.employees;

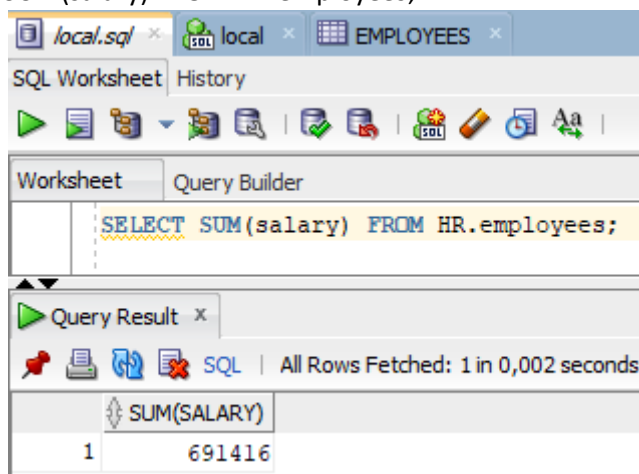


4. Найти минимальные комиссионные сотрудников.
SELECT MIN(commission_pct) FROM HR.employees;



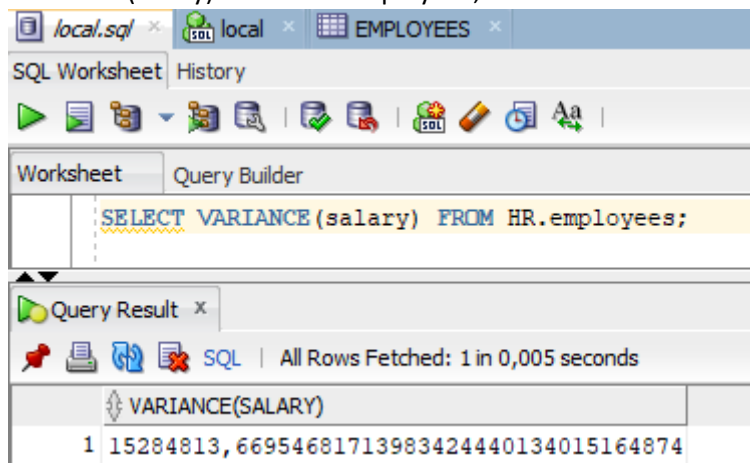
5. Найти сумму, выплачиваемую всем сотрудникам, как заработная плата, без учета комиссионных.

`SELECT SUM(salary) FROM HR.employees;`



6. Найти дисперсию заработных плат сотрудников.

`SELECT VARIANCE(salary) FROM HR.employees;`



7. Найти минимальные и максимальные зарплаты в каждом отделе.

`SELECT department_id, MIN(salary), MAX(salary) FROM HR.employees GROUP BY department_id;`

local.sql x local x EMPLOYEES x

SQL Worksheet History

Worksheet Query Builder

```
SELECT department_id, MIN(salary), MAX(salary) FROM HR.employees GROUP BY department_id;
```

Query Result x

SQL | All Rows Fetched: 12 in 0,002 seconds

	DEPARTMENT_ID	MIN(SALARY)	MAX(SA...)
1	50	2100	8200
2	10	4400	4400
3	20	6000	13000
4	40	6500	6500
5	70	10000	10000
6	110	8300	12008
7	90	17000	24000
8	60	4200	9000
9	100	6900	12008
10	30	2500	11000
11	80	6100	14000
12	(null)	7000	7000

8. Найти среднее значение комиссионных в каждом отделе.

SELECT department_id, AVG(commission_pct) FROM HR.employees GROUP BY department_id;

local.sql x local x EMPLOYEES x

SQL Worksheet History

Worksheet Query Builder

```
SELECT department_id, AVG(commission_pct) FROM HR.employees GROUP BY department_id;
```

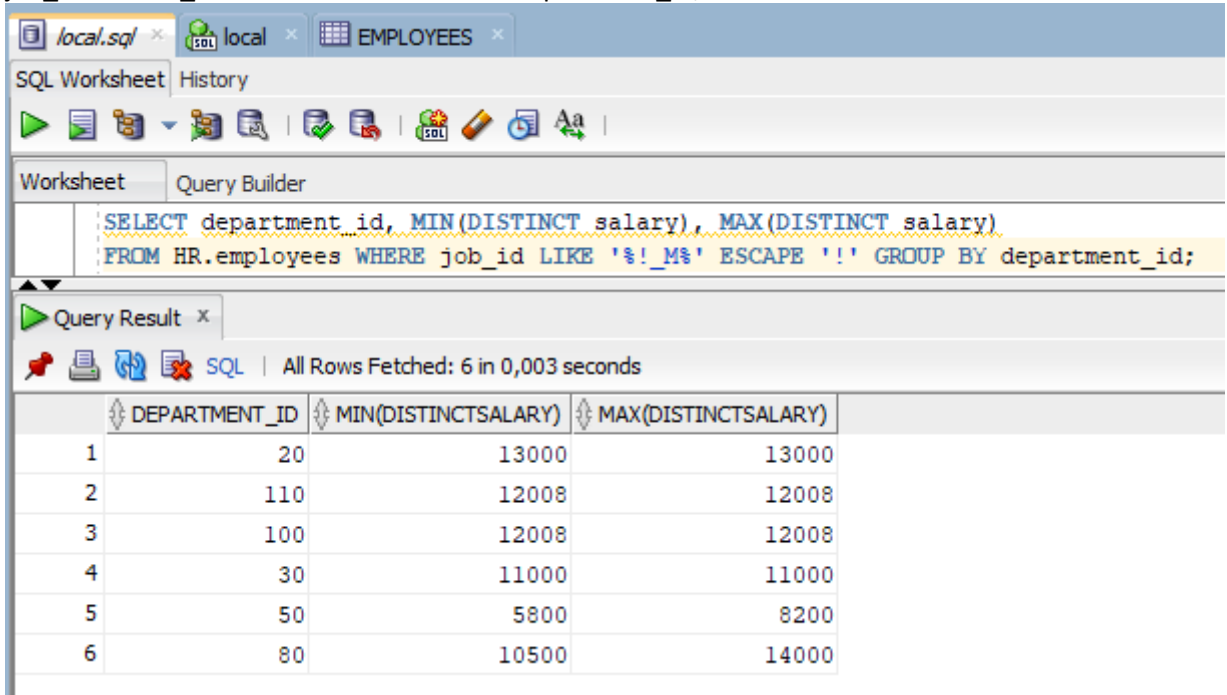
Query Result x

SQL | All Rows Fetched: 12 in 0,003 seconds

	DEPARTMENT_ID	AVG(COMMISSION_PCT)
1	50	(null)
2	10	(null)
3	20	(null)
4	40	(null)
5	70	(null)
6	110	(null)
7	90	(null)
8	60	(null)
9	100	(null)
10	30	(null)
11	80	0,225
12	(null)	0,15

9. Найти минимальные и максимальные зарплаты менеджеров каждого отдела.

SELECT department_id, MIN(DISTINCT salary), MAX(DISTINCT salary) FROM HR.employees WHERE job_id LIKE '%!_M%' ESCAPE '!' GROUP BY department_id;

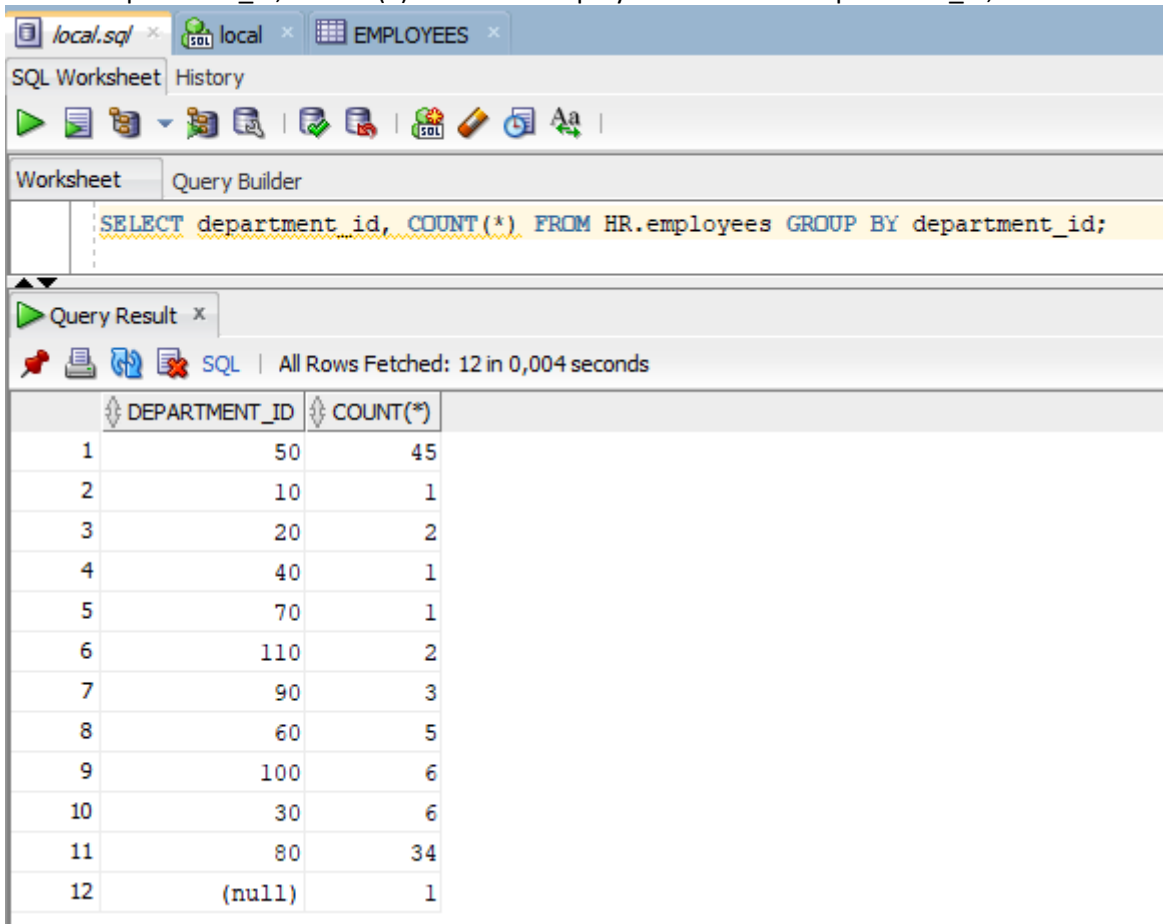


The screenshot shows the SQL Developer interface. The top pane displays the query: `SELECT department_id, MIN(DISTINCT salary), MAX(DISTINCT salary) FROM HR.employees WHERE job_id LIKE '%!_M%' ESCAPE '!' GROUP BY department_id;`. The bottom pane shows the query results in a table format.

	DEPARTMENT_ID	MIN(DISTINCTSALARY)	MAX(DISTINCTSALARY)
1	20	13000	13000
2	110	12008	12008
3	100	12008	12008
4	30	11000	11000
5	50	5800	8200
6	80	10500	14000

10. Найти число сотрудников в каждом отделении

SELECT department_id, COUNT(*) FROM HR.employees GROUP BY department_id;



The screenshot shows the SQL Developer interface. The top pane displays the query: `SELECT department_id, COUNT(*) FROM HR.employees GROUP BY department_id;`. The bottom pane shows the query results in a table format.

	DEPARTMENT_ID	COUNT(*)
1	50	45
2	10	1
3	20	2
4	40	1
5	70	1
6	110	2
7	90	3
8	60	5
9	100	6
10	30	6
11	80	34
12	(null)	1

11. Найти минимальную зарплату каждого отдела, в котором минимальная зарплата меньше 5000.

SELECT department_id, MIN(salary) FROM HR.employees GROUP BY department_id HAVING MIN(salary) < 5000;

14. Найти отделения, в которых минимальная зарплата сотрудников меньше 3000.
 SELECT department_id, MIN(salary) FROM HR.employees GROUP BY department_id HAVING
 MIN(salary) < 3000;

The screenshot shows a SQL Worksheet with the following query: `SELECT department_id, MIN(salary) FROM HR.employees GROUP BY department_id HAVING MIN(salary) < 3000;`. The query result is displayed in a table with 2 rows and 2 columns: DEPARTMENT_ID and MIN(SALARY).

DEPARTMENT_ID	MIN(SALARY)
1	50
2	30

15. Найти среднюю ежегодную зарплату в каждом отделе. Отсортировать по отделам в
 порядке возрастания.
 SELECT department_id, AVG(12*salary) FROM HR.employees GROUP BY department_id ORDER BY
 department_id;

The screenshot shows a SQL Worksheet with the following query: `SELECT department_id, AVG(12*salary) FROM HR.employees GROUP BY department_id ORDER BY department_id;`. The query result is displayed in a table with 12 rows and 2 columns: DEPARTMENT_ID and AVG(12*SALARY).

DEPARTMENT_ID	AVG(12*SALARY)
1	10
2	20
3	30
4	40
5	50
6	60
7	70
8	80
9	90
10	100
11	110
12	(null)

16. Найти количество сотрудников в каждом отделении, которых наняли позже 01.01.06.
 Отсортировать по отделениям в порядке убывания.
 SELECT department_id, COUNT(employee_id) FROM HR.employees WHERE hire_date > '01.01.06'
 GROUP BY department_id ORDER BY department_id DESC;

local.sql x local x JOB_HISTORY x

SQL Worksheet History

Worksheet Query Builder

```
SELECT department_id, COUNT(employee_id) FROM HR.employees
WHERE hire_date > '01.01.06' GROUP BY department_id ORDER BY department_id DESC;
```

Query Result x

SQL | All Rows Fetched: 6 in 0,002 seconds

	DEPARTMENT_ID	COUNT(EMPLOYEE_ID)
1	(null)	1
2	100	2
3	80	19
4	60	4
5	50	26
6	30	2

17. Найти id сотрудников, у которых есть хотя бы один подчиненный, и количество подчиненных. Отсортировать по возрастанию числа подчиненных.

```
SELECT manager_id, COUNT(employee_id) AS "Число подчиненных" FROM HR.employees GROUP BY
manager_id ORDER BY COUNT(employee_id);
```


local.sql x local x EMPLOYEES x

SQL Worksheet History

Worksheet Query Builder

```
SELECT manager_id, COUNT(employee_id) AS "Число подчиненных" FROM HR.employees
GROUP BY manager_id ORDER BY COUNT(employee_id);
```

Query Result x

SQL | All Rows Fetched: 19 in 0,004 seconds

	MANAGER_ID	Число подчиненных
1	102	1
2	(null)	1
3	201	1
4	205	1
5	103	4
6	101	5
7	114	5
8	108	5
9	148	6
10	146	6
11	145	6
12	149	6
13	147	6
14	124	8
15	122	8
16	121	8
17	120	8
18	123	8
19	100	14